ADEPT — TCN Base Learner

Phase 3 Results: Analysis and Next Steps

Guilherme Leitão

Master Thesis — IST / INESC-ID

May 3, 2026

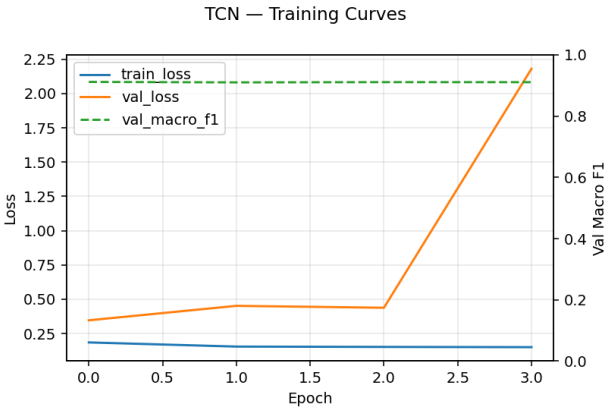
What was trained

- **Task:** same binary classification setting as RF Phase 2 — same 4-attack dataset, same flow-level 70/15/15 split (seed=42), same adversarial exclusions (fuzzing / portscan / syn_dos).
- **Input:** `sequence.npy` per source — each sample is a $[T{=}10,\ F{=}1090]$ window (1 088 nPrint header bits + IPAT z-score + Direction). Lazy-loaded via memmaps; ∼17 GB per source.
- **Architecture:** 3-block causal TCN, channels $[128, 128, 128]$, kernel 3, dilations $\{1, 2, 4\}$, receptive field $29 \gg T$, head: mean+max pooling → linear. **806 018 parameters.** AMP enabled (CUDA).
- **Training:** AdamW ($\eta{=}10^{-3}$, wd$= 10^{-4}$), cosine LR, `class_weight=balanced`, early stopping patience 3 on val macro-F1.

Headline metrics (test, overall)

Macro F1	ROC AUC	PR AUC	Recall (adv)	Precision (adv)
0.913	**0.974**	0.985	0.854	**0.999**

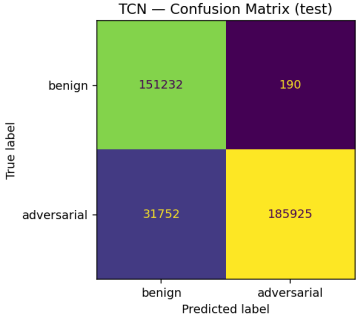
Training behaviour



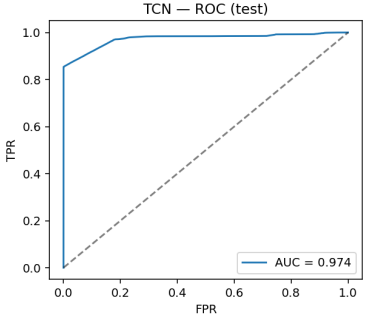
Ep.	Tr. loss	Val. loss	Val. F1
0	0.185	0.347	**0.912**
1	0.155	0.452	0.910
2	0.153	0.438	0.911
3	0.151	**2.181**	0.911

- **Best checkpoint: epoch 0** (first pass through data). Val macro-F1 never improved thereafter.
- Train loss decreases while val loss climbs — early overfitting. Val-loss spike at epoch 3 is consistent with AMP overconfidence: class probabilities approach 0/1, driving log-loss up while argmax F1 stays flat.
- Early stopping (patience=3) triggered correctly; saved checkpoint is unaffected.

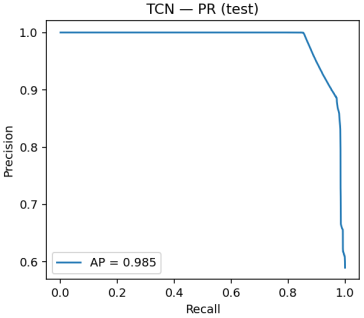
Confusion matrix & curves



Confusion Matrix



ROC (AUC = 0.974)



Precision–Recall (AP = 0.985)

- High adversarial *precision* (0.999): when the TCN fires, it is almost never wrong — very few false positives.
- Moderate adversarial *recall* (0.854): 14.6% of adversarial windows are missed — mainly IAT-only and Mirai variants (see next slides).

Per-variant performance (vs. balanced benign)

Variant	$F1_{adv}$	ROC AUC	PR AUC
ddos_adv_both / size	0.994 / 0.992	1.000 / 0.999	1.000 / 0.998
ddos_adv_iat	**0.401**	0.956	**0.697**
dos_slow_adv_both / size	0.997 / 0.997	1.000	1.000
dos_slow_adv_iat	**0.797**	0.960	**0.880**
ssdp_flood_adv_{both, iat, size}	0.995 / 0.993 / 0.995	1.000–0.9997	1.000–0.999
mirai_adv_both / size	0.376 / 0.473	0.587 / 0.627	0.324 / 0.405
mirai_adv_iat	**0.000**	0.562	**0.110**

- **SSDP Flood:** near-perfect across all modes — header patterns uniquely identify TANTGen perturbations.
- **DDoS / DoS Slow IAT:** recall collapses when only timing is perturbed — the single IPAT z-score is drowned by 1 088 unchanged header bits.
- **Mirai:** low F1 across all modes; IAT variant is a complete failure (0 adversarial predictions out of 2 928).

Same-base vs. balanced benign: where detection is harder

Variant	F1 (balanced)	F1 (same-base)	Δ
ddos_adv_both / size	0.994 / 0.992	1.000 / 0.998	+0.003–+0.006
ddos_adv_iat	0.401	0.405	+0.004
dos_slow_adv_both / size	0.997 / 0.997	0.999 / 0.999	+0.001–+0.002
dos_slow_adv_iat	0.797	0.799	+0.002
ssdp_flood_adv_{both, iat, size}	0.995–0.997	0.997	≈ 0
mirai_adv_both / size	0.376 / 0.473	0.380 / 0.476	≈ 0
mirai_adv_iat	0.000	0.000	0

- **Volumetric attacks:** same-base F1 $\geq$ balanced F1 — TANTGen's payload / IPAT shifts are prominent even against same-source benign.
- **Mirai / IAT variants:** unlike the RF, the TCN does *not* show same-base bleed; F1 is uniformly weak regardless of benign pool — the TCN simply cannot discriminate these variants, independent of benign mix.

RF vs. TCN: head-to-head ($F1_{adv}$, balanced benign)

Variant	RF	TCN	Winner
ddos_adv_both / size	0.91 / 0.91	**0.994 / 0.992**	**TCN (+0.08)**
ddos_adv_iat	**0.90**	0.401	**RF (+0.50)**
dos_slow_adv_both / size	0.955 / 0.955	**0.997 / 0.997**	**TCN (+0.04)**
dos_slow_adv_iat	**0.953**	0.797	**RF (+0.16)**
ssdp_flood_adv_both / iat / size	0.91	**0.993–0.995**	**TCN (+0.08)**
mirai_adv_both / iat / size	**0.887 / 0.883 / 0.761**	0.376 / 0.000 / 0.473	**RF (+0.3–0.9)**

- **TCN dominates on size/both modes** — nPrint header bits encode payload padding directly; 3 blocks of causal convolutions detect TANTGen's structural changes better than RF aggregates.
- **RF dominates on IAT-only modes and Mirai** — aggregated IPAT statistics (mean/std/min/max) give IAT a 16.7% feature share; the TCN's single IPAT z-score at 0.09% is swamped.
- **The two models are complementary** — the meta-learner can route by perturbation mode.

Why IAT modes fail: feature geometry

TCN input: 1 090 features per packet

- 1 088 nPrint header bits (IP flags, TTL, ports, TCP flags, payload size bytes, ...)
- 1 IPAT z-score (*the only IAT signal*)
- 1 Direction bit

IPAT share $= 1/1090 \approx \mathbf{0.09}\%$

RF input: 24 aggregate features per window

- 4 IPAT statistics: mean, std, min, max
- 20 other aggregates (packet size, payload, TCP flags, ...)

IPAT share $= 4/24 \approx \mathbf{16.7}\%$

Consequence

IAT-only perturbations change exactly 1 out of 1 090 features per packet. The convolutional filters learn to rely on the 1 088 unchanged header bits (which strongly identify the traffic type), rendering the IPAT z-score invisible.

This also explains the Mirai IAT complete failure (0 detections): Mirai headers are particularly uniform within a capture, so the TCN confidently predicts *benign* for every window — it never sees enough contrast in the header bits to trigger on the timing signal.

Fix candidates for future work:

- Drop nPrint bits; use IPAT-only sequence (matches RF's focus).
- Feature groups with separate convolution streams.
- Add explicit IPAT features at multiple lags.

Conclusions

1. **SSDP Flood and DoS Slow (both/size modes) are detected near-perfectly ($F1_{adv} \geq 0.993$)** — causal convolutions over nPrint sequences capture payload-level TANTGen perturbations better than RF aggregates.
2. **TCN outperforms RF on all size/both modes by 4–8 F1 points** — structural header changes are directly visible in the raw packet sequence.
3. **IAT-only modes are the TCN's Achilles heel** — IPAT accounts for 0.09% of the input; the model ignores it. DDoS IAT F1 drops to 0.401; DoS Slow IAT to 0.797.
4. **Mirai is entirely missed**: $F1_{adv}$ 0.37–0.47 for both/size, **0.000** for IAT — the most critical limitation of this run.
5. **Training converged in a single epoch** (best_epoch=0); the val-loss spike at epoch 3 is consistent with AMP overconfidence but does not affect the saved checkpoint.
6. **RF and TCN are strongly complementary** — their weak spots are disjoint, motivating the meta-learner fusion in Phase 4.

Next steps

Short-term (End of May) — complete the ensemble & tune end-to-end

1. **Phase 4 — Mini-Transformer meta-learner** fusing RF + TCN logits via self-attention. Expected to recover Mirai / IAT variants where RF is stronger.
2. **TCN feature investigation** — ablation: IPAT-only sequence vs. full nPrint. If IPAT-only closes the Mirai gap, feed it as a separate stream.
3. **Sync with Inês** on TANTGen version in use — confirm Fuzzing snaplen-200B no-op and whether Mirai IAT perturbation magnitude is configurable.
4. **DPSS hyperparameter sweep** — grid-search `window_size` $\in \{5, 10, 20\}$, TCN channels/depth, meta-learner params. Objective: maximize ensemble detection accuracy.

Long-term — defense layer & write-up

5. **Phase 5 — Purification Transformer** (decoder-only Seq2Seq) projecting adversarial windows back onto the benign manifold; KV-caching for latency.
6. **Phase 6 — RL Adaptation Agent** re-weighting the ensemble online from TANTGen feedback + offline DPI validation.
7. **Dissertation write-up.**

Reproducibility

Artifacts under `ADEPT/data/models/tcn/`

- `tcn.pt` — best model state_dict (epoch 0)
- `metrics.json` — overall + per-source (vs_all / vs_balanced / vs_same_base) + per-`label_str`
- `split.npz`, `dataset_index.npz` — exact train/val/test partition (shared with RF via `assemble_index`)
- `training_info.json`, `training_curves.json`, `tcn_config.json` — full training trace and resolved config
- `*.png` — confusion matrix, ROC, PR, training curves

Re-run from scratch

```
python3 scripts/adversarial_generation/generate_adversarials.py
python3 scripts/packet_to_tensor/process_all_attacks.py --tantgen
python3 scripts/tcn/run_tcn.py
```